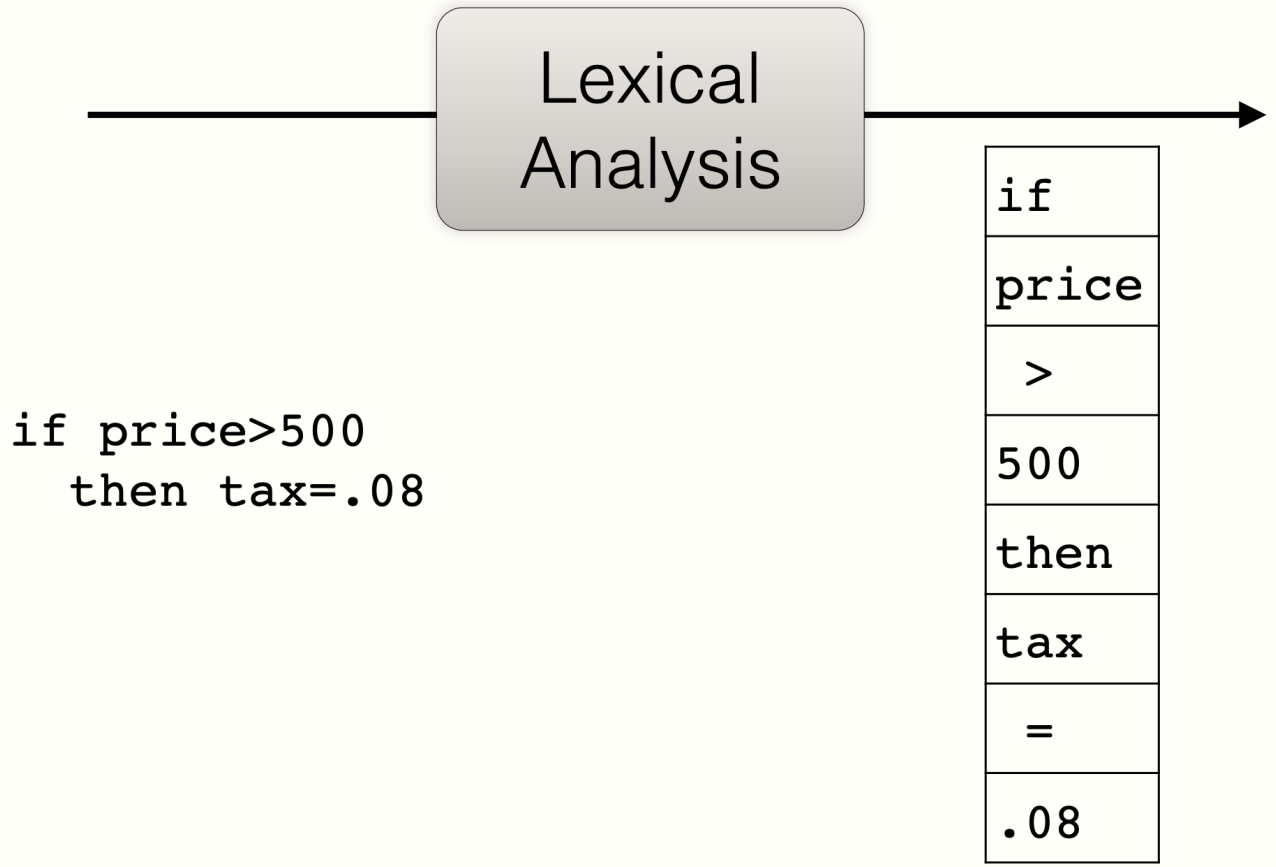
《编译原理》

第11讲：词法分析

何冬杰
重庆大学

Lexical Analysis

- breaks input sequence of characters into individual words, aka “tokens”



Lexical Tokens

- A language classifies lexical tokens into token types
  - So, a token type specifies a set of acceptable tokens.
  - Reserved words (E.g., IF, RETURN) are tokens that cannot be used as identifiers

Type	Examples
ID	`foo n14 last`
NUM	`73 0 00 515 082`
REAL	`66.1 .5 10. 1e67`
IF	`if`
COMMA	`,`
NOTEQ	`!=`
LPAREN	`(`

- Example 1: **`if (price>500) then tax=.08`**
  - the lexical analysis returns the sequence of tokens

```
IF LPAREN ID(price) GT NUM(500) RPAREN THEN ID(tax) EQ REAL(0.08)
```

Lexical Analysis: Examples

- Example 2: `if (price>500) then tax=1xab`
  - Lexical analysis returns **ERROR** because 1xab is neither a number nor an identifier
  - Lexical analysis can help in reporting where an error occurs in the code
    - By recognizing '\n' as a token and incrementing the line number

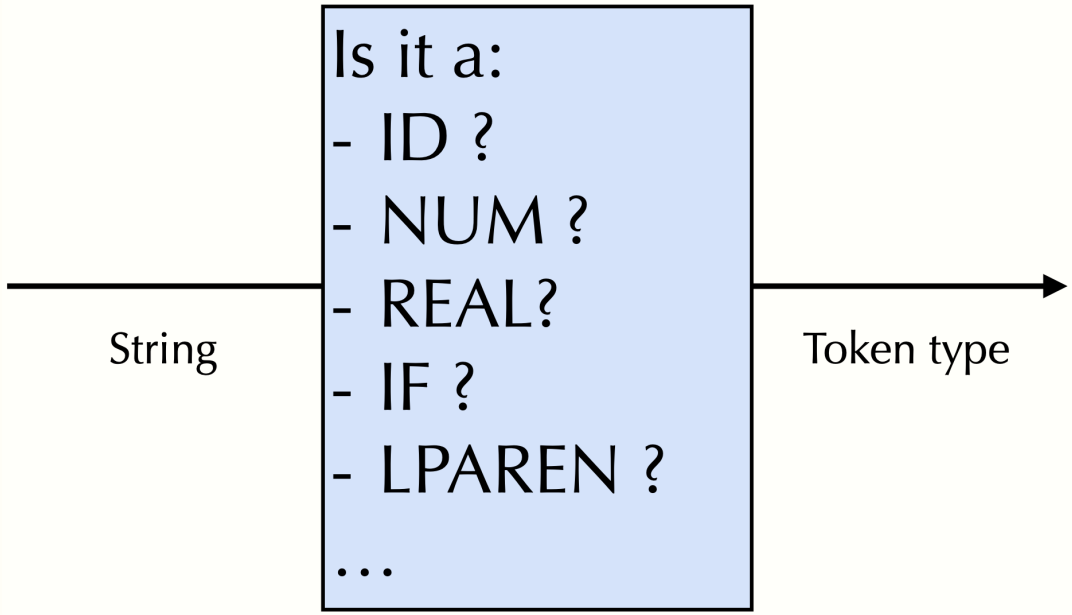
- Example 3: `if (price>500) thn tax=.08`
  - The lexical analysis returns:

```
IF LPAREN ID(price) GT NUM(500) RPAREN ID(thn) ID(tax) EQ REAL(0.08)
```

  - Is this an error at the level of lexical analysis?
    - No, it is an error at the level of syntax analysis (next lectures)!

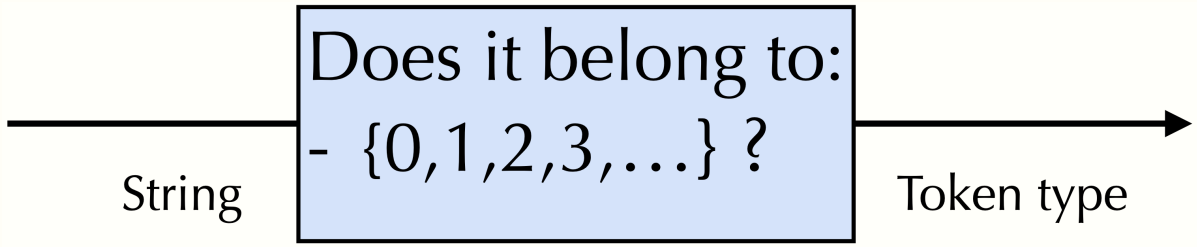
Towards Implementing a Lexical Analysis

- Recall: Lexical analysis breaks input into tokens.
- The lexical analysis needs to decide the token type for a given string (i.e., sequence of characters).



A Set Membership Question

- Recall: a token type specifies a set of acceptable tokens (i.e., strings).
  - The set of acceptable tokes for NUM is {0,1,2,3,…}.



  - But this set is **infinite**…
- How can we mechanically decide if a string belongs to a (possibly infinite) set S of strings?
  - Use a finite representation of S
    - **Regular expressions**
  - Check whether such a finite representation accepts the string
    - **Deterministic finite-state automata**

Regular Expressions (RE)

Syntactic Sugar

- [0-9] shorthand for 0 | 1 | … | 9
- R? shorthand for (R | ε) (i.e., R is optional)
- R+ shorthand for (R R*) (i.e., at least one R)

❑RE Grammar

∅ (matches no string)	Concatenation $(R_1\ R_2)$
ε (epsilon – matches empty string)	Alternation $(R_1 \mid R_2)$
Literals (‘a’, ‘b’, ‘2’, ‘+’, etc.) drawn from alphabet	Kleene star $(R\ *)$

❑RE Semantics: each regular expression represents a set of strings

- $[\![\emptyset]\!] = \{\};\ [\![\varepsilon]\!] = \{\text{“”}\};\ [\![\text{’a’}]\!] = \{\text{“a”}\}$
- $[\![R_1\ R_2]\!] = \{s \mid s = \alpha\,\hat{}\,\beta \text{ and } \alpha \in [\![R_1]\!] \text{ and } \beta \in [\![R_2]\!]\}$
- $[\![R_1 \mid R_2]\!] = \{s \mid s \in [\![R_1]\!] \text{ or } s \in [\![R_2]\!]\} = [\![R_1]\!] \cup [\![R_2]\!]$
- $[\![\text{R}\ *]\!] = [\![\epsilon \mid \text{RR}\ *]\!] = \{s \mid s = \text{“”} \text{ or } s = \alpha\hat{}\beta \text{ and } \alpha \in [\![\text{R}]\!] \text{ and } \beta \in [\![\text{R}\ *]\!]\}$

❑Examples:

$(\ 0 \mid 1\ )*\ 0$	Binary numbers that are multiples of 2
$b*(abb*)*(a\|\varepsilon)$	Strings of a’s and b’s without consecutive a’s
$(a\|b)*aa(a\|b)*$	Strings of a’s and b’s with consecutive a’s

More RE Examples

- Σ = {a, b}
- L(a | b) = {a, b}
- L((a | b) (a | b)) = {aa, ab, ba, bb}
- L(a*) = {ε, a, aa, aaa, aaaa, …}
- L((a | b)*) = {ε, a, b, aa, ab, ba, bb, aaa, aab, …}
- L(a | a*b) = {a, b, ab, aab, aaab, …}

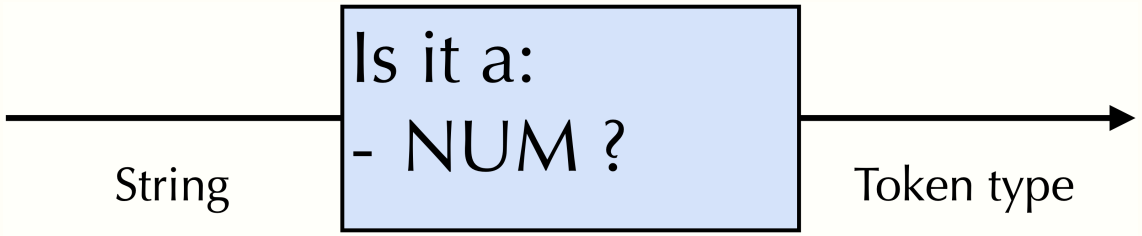
Regular Expressions to Specify Token Types

Reg Exp	Token Type
`if`	IF
`[a-z][a-z0-9]*`	ID
`[0-9]+`	NUM
`([0-9]+ "." [0-9]*) |` `([0-9]* "." [0-9]+)`	REAL

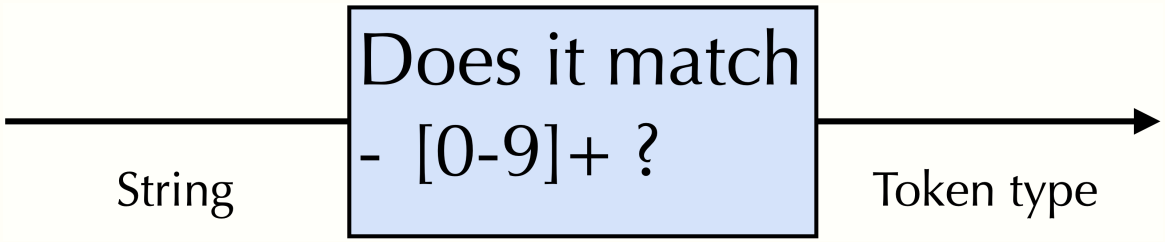
- Question: What is the token type of input "iffy"?
  - We want the token **ID(iffy)** rather than **IF**.
- In general, we want the longest match:
  - The longest initial substring of the input that can match a regular expression is taken as the next token

Set Membership Question to Matching Question

- Recall: Lexical analysis breaks input into tokens.
- The lexical analysis needs to decide the token type for a given string (i.e., sequence of characters).
- From a Set Membership Question:

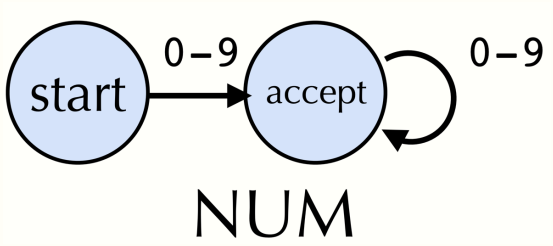


- To a Matching Question:

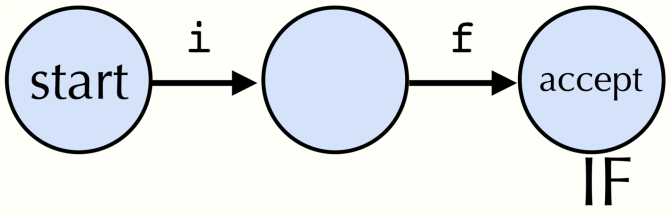


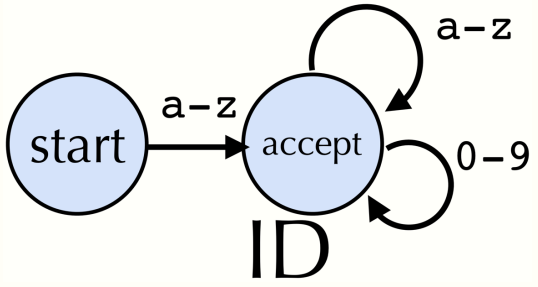
From RE to DFA

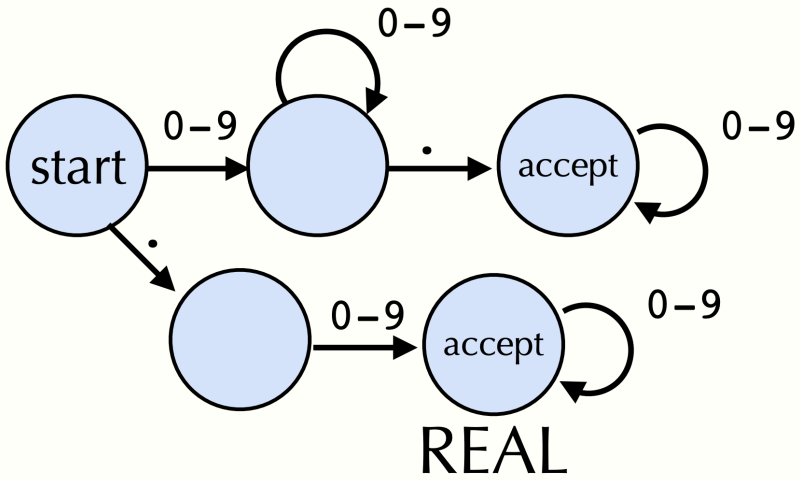
- A **Deterministic Finite-state Automaton** (**DFA**) can be used to decide whether an input matches a regular expression.
- Examples:



Reg Exp	Token Type
`if`	IF
`[a-z][a-z0-9]*`	ID
`[0-9]+`	NUM
`([0-9]+ "." [0-9]*) | ([0-9]* "." [0-9]+)`	REAL



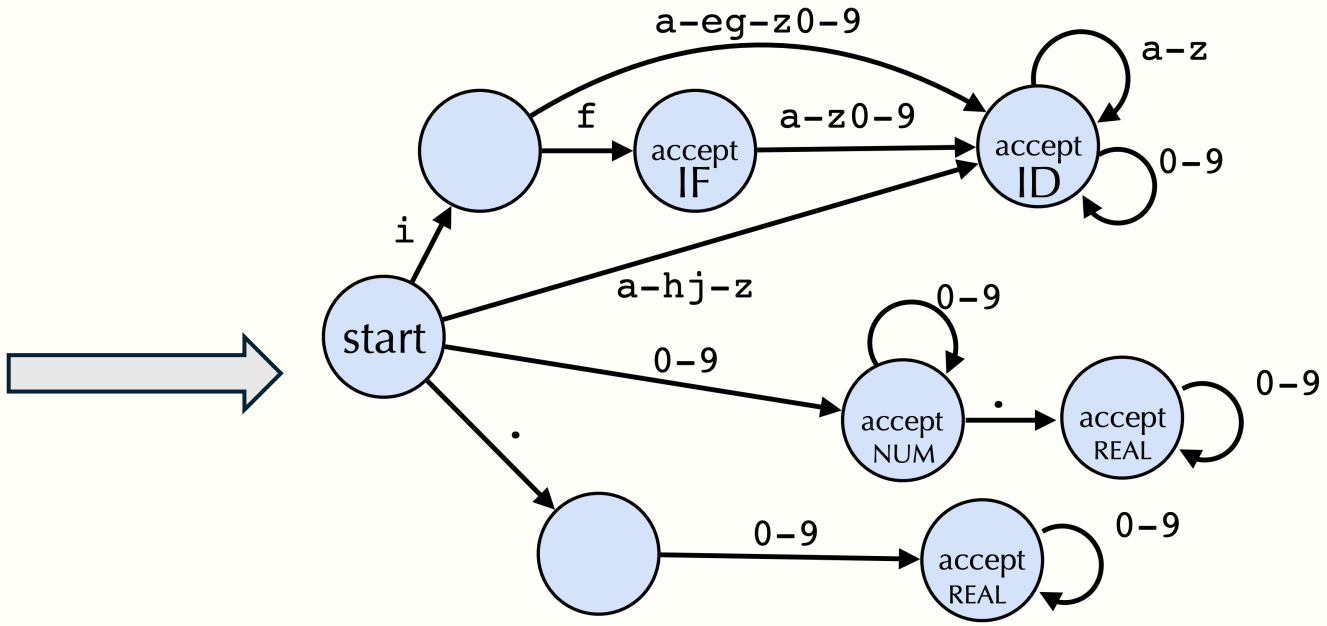




Using DFAs for Lexical Analysis

❑Combined Finite Automaton

Reg Exp	Token Type
`if`	IF
`[a-z][a-z0-9]*`	ID
`[0-9]+`	NUM
`([0-9]+ "." [0-9]*) | ([0-9]* "." [0-9]+)`	REAL



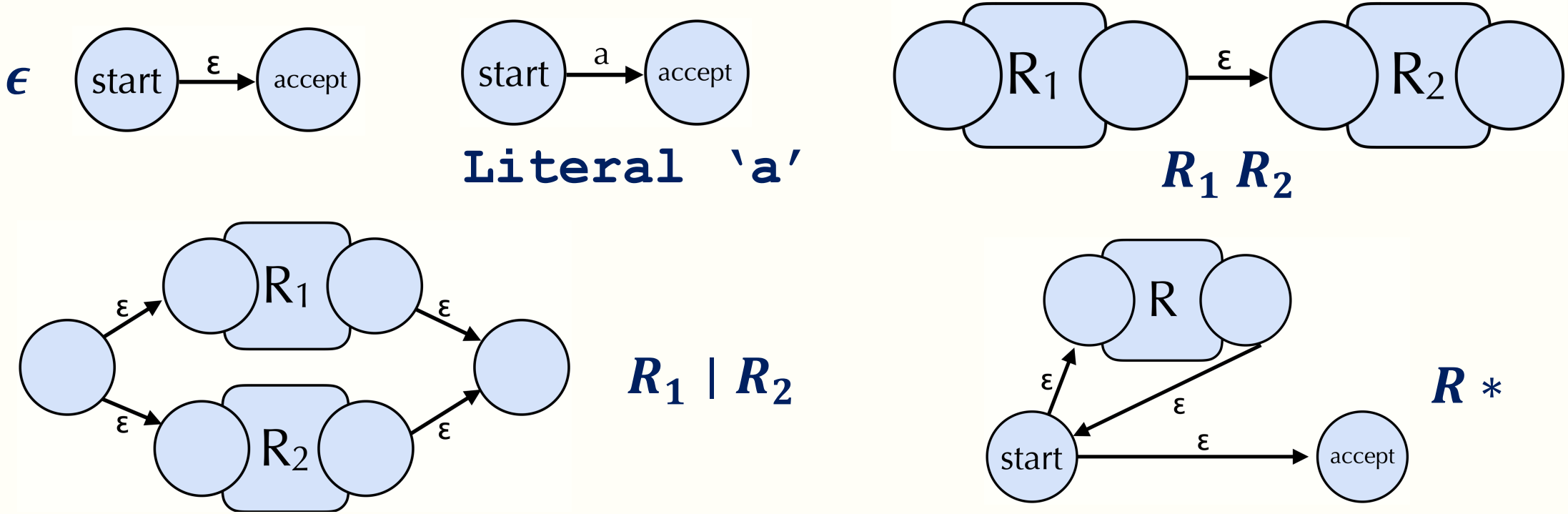
- This DFA takes as input a sequence of characters and returns a Token Type (if the input is accepted). So, this DFA can be used for Lexical Analysis.

❑Using DFAs: usually record the transition function as an array indexed by state and characters (i.e., transition table):

- $\sigma(s_1, i) = s_2$: If the current state is $\boldsymbol{s_1}$ and the input is $\boldsymbol{i}$, transition to $\boldsymbol{s_2}$.

How to convert a RE into a DFA?

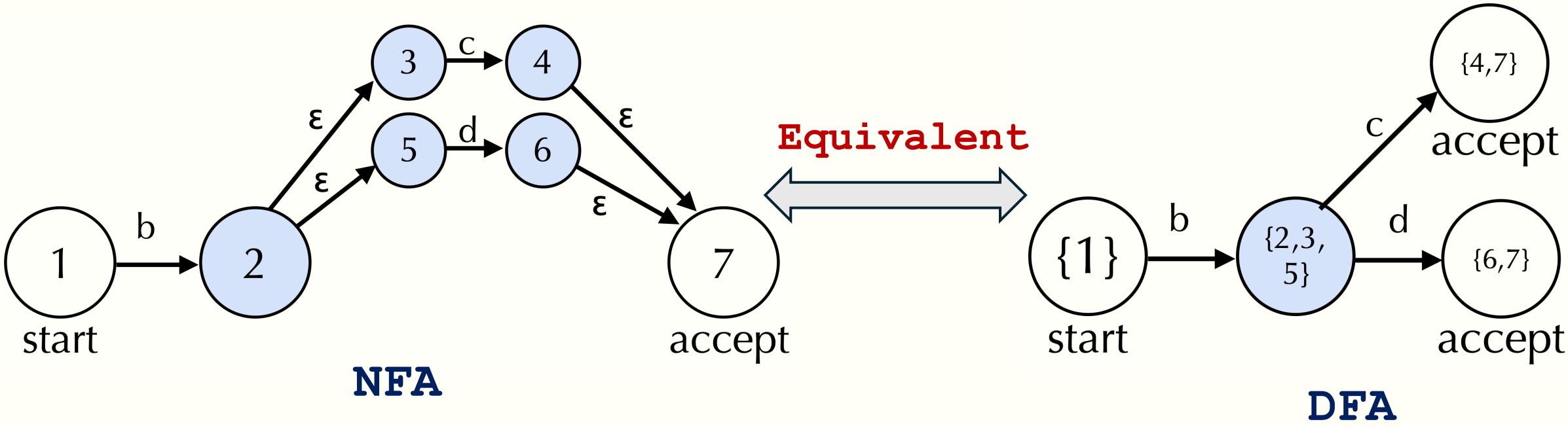
- Step 1: Convert RE to a **Nondeterministic Finite-state Automaton** (**NFA**)



- Step 2: Convert NFA to DFA
  - NFAs are harder to execute and less efficient than DFAs due to nondeterminism

NFA to DFA conversion (intuition)

- **Basic idea**:
  - Each state in the DFA represents a set of states of the NFA under the same inputs
- Examples:



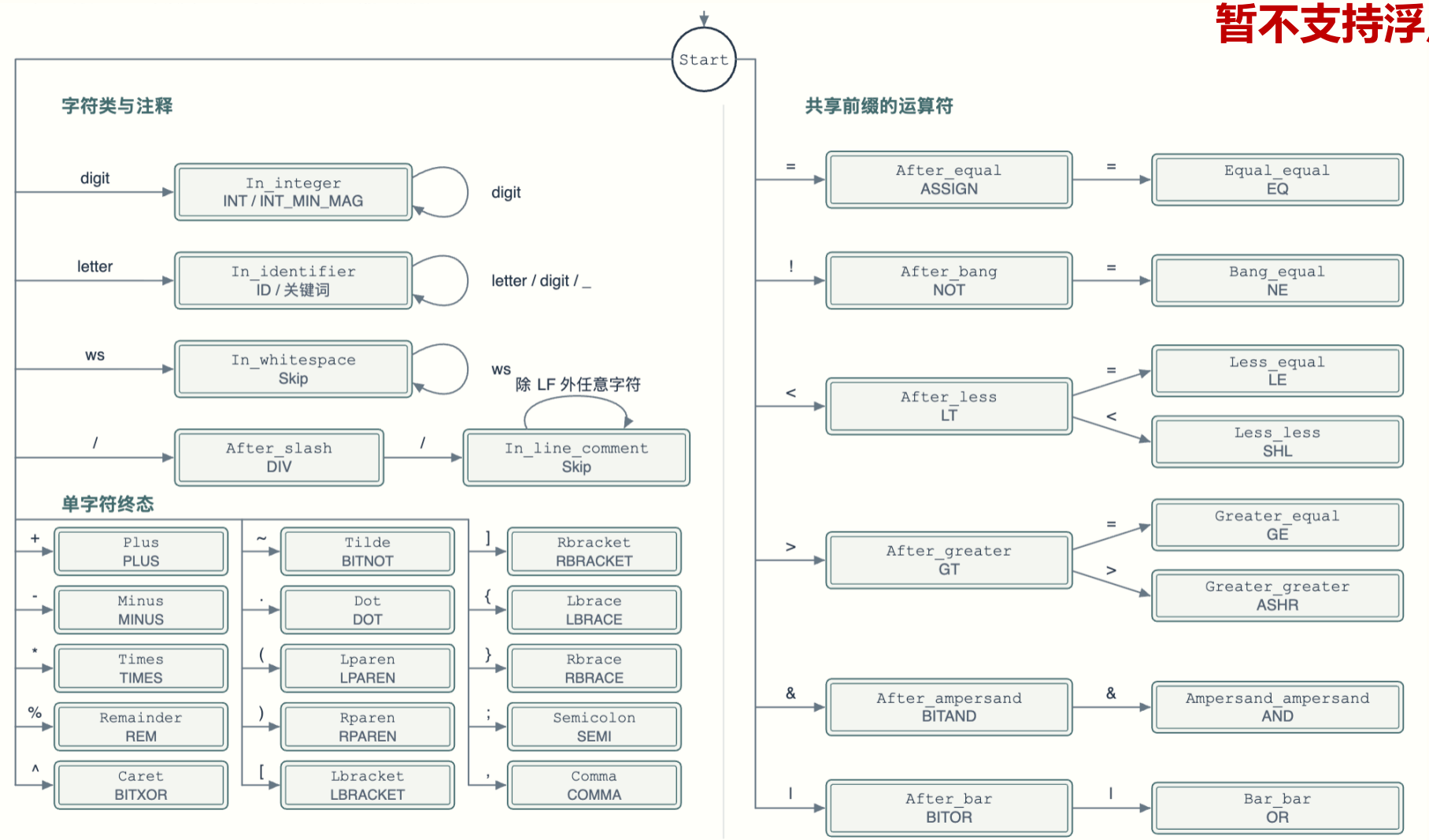
Lexical Analysis Summary

- Use a regular expression R_i to specify the set strings for each token type
  - Example: [0-9]+ specifies the set of strings for NUM
- Construct the NFA formed by $(R_1 \mid R_2 \mid \ldots \mid R_n)$
- Construct the DFA for this NFA
- Produce the transition table for that DFA
- Implement longest match

- Two **methods** for lexical analysis:
  - Use a Lexer generator
  - Write a Lexer by hand (see file lexer.ml)

Lexer.ml中的完整DFA

暂不支持浮点数



双框 = 接受态；第一行是代码状态名，第二行是接受动作。　　digit = 0-9；letter = A-Z / a-z；ws = 空格 / TAB / CR / LF

未列转移 = None：停止扩展，提交最后接受结果；若从未接受则报错。每个词素结束后，扫描驱动重新从 Start 开始。

关键词分类、整数范围检查及行号维护在 DFA 外执行；EOF 由外层追加。本图尚未加入 FLOAT。

实验：识别浮点数Tokens

- 目前，提供的手写**Lexer.ml**脚手架程序中已经能够识别Lang语言的所有类型的Tokens。接下来，请同学们拓展这个词法分析器，支持识别浮点数类型的Tokens
  - 支持的浮点数对应正则表达式为： [0-9]+[.][0-9]+([eE][+-]?[0-9]+)?

- 任务：
  - 将FLOAT正则表达式转为NFA
  - 用子集构造法将NFA转为DFA
  - 最小化DFA（选做）
  - 将DFA接入现有Lexer，使其能够识别浮点数